

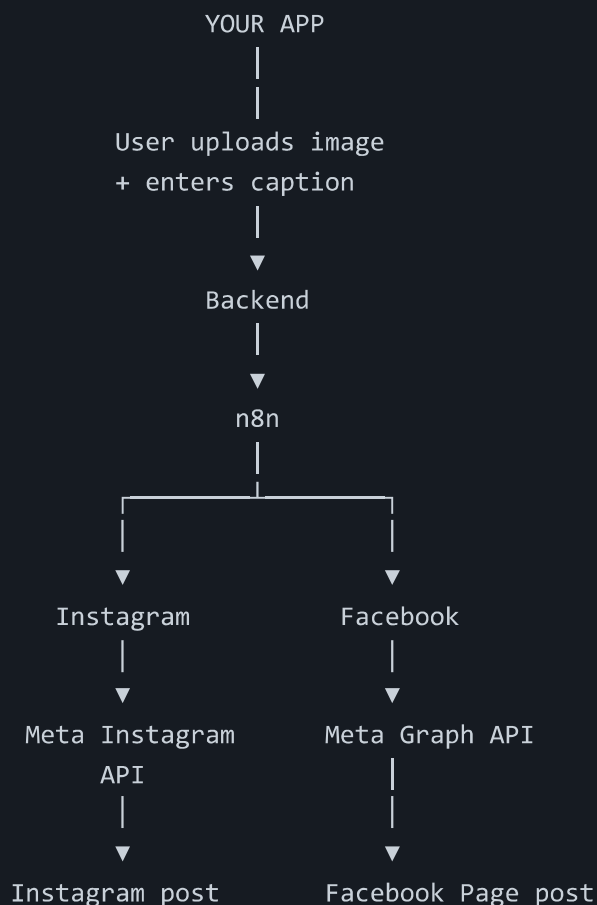
Absolutely. You now have **both Instagram and Facebook working end-to-end**, so here is the complete setup as a beginner-friendly “ABCD” tutorial from zero up to the first successful API post.

I’ll separate **what you configured, why you configured it, where to click, what token you get, what each ID means, and how you tested the first post.**

One important note: Meta's current official Postman collections are updated frequently. The guide below is based on the Meta dashboard flow you actually completed plus Meta's current official Postman documentation. Meta's current Instagram Login setup requires `instagram_business_basic` and `instagram_business_content_publish` for publishing; the Facebook/Page flow uses a User Access Token to obtain a Page Access Token. ([Postman]^[1])

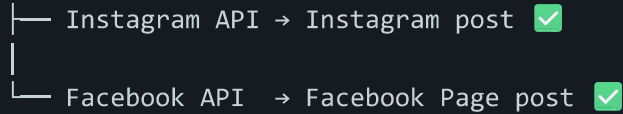
PART 0 — Understand the whole system first

Your application will eventually work like this:



For your first test, however, you did this manually:

Postman



That was the correct approach because it proves **Meta works before n8n is introduced**.

PART 1 — Create the Meta Developer App

1. Open Meta Developers

Use:

<https://developers.facebook.com/>

Then:

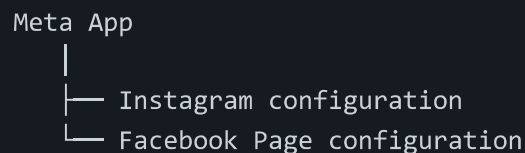
My Apps → Create App

You created your app:

VelorQ

This app is the central container for your API integrations.

Think:



PART 2 — Instagram setup

You chose:

Instagram API → API setup with Instagram Login

This is important because this is the Instagram flow you actually configured.

Meta's current official Postman collection describes the Instagram Login setup as supporting Instagram Professional accounts and lists these publishing-related permissions:

```
instagram_business_basic
instagram_business_content_publish
```

It also lists messaging/comment permissions for those separate capabilities. ([Postman]^[1])

PART 3 — Make Instagram Professional

Your Instagram account had to be a Professional account.

Conceptually:

```
Instagram account
  ↓
Professional account
  ↓
Instagram API
```

You used the same Instagram account you created for testing:

```
@velar.q
```

That was correct.

PART 4 — Add Instagram API use case

Inside the Meta app:

```
Use cases
  ↓
Manage messaging & content on Instagram
```

You selected the Instagram use case.

The Meta screen then showed:

Welcome to the Instagram API

and the setup sections.

PART 5 — Add Instagram permissions

Inside:

```
Instagram API
↓
API setup with Instagram Login
↓
Permissions and features
```

You added/verified:

```
instagram_business_basic
instagram_content_publish
```

For your current Instagram Login flow, the current official Meta Postman collection uses the newer permission name:

```
instagram_business_content_publish
```

along with:

```
instagram_business_basic
```

for the Instagram Login API. ([Postman]^[1])

Your dashboard showed the publishing capability as:

```
instagram_content_publish
```

and it became:

```
Ready for testing
```

That was the key permission that allowed your publishing test to work.

PART 6 — Add Instagram Tester

You went to:

App roles

and added:

@velar.q

as:

Instagram Tester

Then the Instagram account accepted the invitation.

The flow was:

```
Meta App
  ↓
Instagram Tester
  ↓
@velar.q
  ↓
Accept invitation
```

This is required for testing an app against a non-live account in this setup.

PART 7 — Add Instagram account

Then you returned to:

```
Instagram API
  ↓
API setup with Instagram Login
  ↓
Generate access tokens
```

and selected:

@velar.q

Meta displayed the account and its numeric Instagram ID.

The important concept is:

```
Instagram username:  
@velar.q
```

```
Instagram User ID:  
17841433886128760
```

The API uses the **numeric ID**, not the username.

That mistake caused your first API error:

```
Object with ID 'velar.q' does not exist
```

The fix was:

```
17841433886128760
```

PART 8 — Generate Instagram access token

Under:

```
Generate access tokens
```

you clicked:

Generate token

Meta generated an Instagram access token for the connected account.

Conceptually:

```
Instagram Account  
  ↓  
Instagram Access Token
```

Do not think of the token as your app password.

It represents authorized API access for the relevant Instagram account/app relationship.

And **never expose it publicly**.

PART 9 — Validate Instagram token

You opened Meta's API Integration Helper.

It showed:

```
Enter your access token
```

and:

Validate

You pasted the token and validated it successfully.

The page also showed:

```
@velar.q  
17841433886128760
```

That proved the token could be associated with the Instagram account.

PART 10 — Ignore the “Hello World” messaging tester

You were initially confused because the API helper displayed:

```
Send messages  
Message type: Text  
Custom message: Hello World
```

That was **not** the Instagram publishing API.

It was the messaging helper.

So:

```
Hello World  
↓  
Messaging API
```

not:

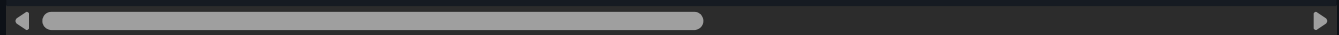
```
Hello World
↓
Publishing API
```

You correctly skipped it.

PART 11 — Create the first Instagram post

You used this public image:

```
https://images.unsplash.com/photo-1780969905523-b0689163243e?q=80&w=465&auto=format&fit
```



This is important because Meta needs to be able to fetch the image from a publicly reachable URL.

Your local file:

```
C:\Users\you\photo.jpg
```

would not be sufficient for the server-to-server publishing request.

Instead:

```
Public HTTPS URL
```

was used.

PART 12 — Instagram Request 1: Create media container

You created this Postman request.

Method

```
POST
```

URL

```
https://graph.instagram.com/v25.0/17841433886128760/media
```

Body

Postman:

```
Body  
→ x-www-form-urlencoded
```

Fields:

```
image_url  
caption  
access_token
```

Example:

```
image_url =  
https://images.unsplash.com/photo-1780969905523-b0689163243e?q=80&w=465&auto=format&fit
```

```
caption =  
My first Instagram API test
```

```
access_token =  
YOUR_INSTAGRAM_TOKEN
```

The successful response was:

```
{  
  "id": "18101283815129538"  
}
```

That `id` is a **media container ID**.

It is not yet the final Instagram post ID.

PART 13 — Instagram Request 2: Check container status

You then made:

Method

```
GET
```

URL

```
https://graph.instagram.com/v25.0/18101283815129538
```

Query parameters:

```
fields=status_code,status  
access_token=YOUR_INSTAGRAM_TOKEN
```

Importantly, you fixed a Postman problem here.

For the GET request:

```
Params
```

should contain those parameters.

Not:

```
Body
```

And you used:

```
Authorization → No Auth
```

The successful response:

```
{  
  "status_code": "FINISHED",  
  "status": "FINISHED",
```

```
"id": "18101283815129538"  
}
```

means the container is ready.

PART 14 — Instagram Request 3: Publish

Then you made:

Method

```
POST
```

URL

```
https://graph.instagram.com/v25.0/17841433886128760/media_publish
```

Body:

```
creation_id = 18101283815129538  
access_token = YOUR_INSTAGRAM_TOKEN
```

Successful response returned an Instagram media ID.

You verified that the post actually appeared on:

```
@velar.q
```

🎉 Instagram API publishing was proven.

PART 15 — Instagram flow you should memorize

This is the entire Instagram publishing sequence:

```
IMAGE URL  
↓
```

```
POST /{ig-user-id}/media
↓
container ID
↓
GET /{container-id}?fields=status_code,status
↓
FINISHED
↓
POST /{ig-user-id}/media_publish
↓
Instagram Post
```

Meta's current official collection documents the same container/status/publish model for Instagram publishing. ([Postman]^[2])

PART 16 — Facebook setup

Now we move to Facebook.

Your Facebook target is **not your personal profile**.

It is:

```
Facebook Page
```

Your test Page was:

```
my social publisher
```

with Page ID:

```
1315245281672428
```

PART 17 — Facebook use case

Inside your Meta app:

```
Use cases
↓
Manage everything on your Page
```



Customize

This opened:

Permissions and features

PART 18 — Facebook permissions

You enabled/verified:

```
pages_manage_posts  
pages_read_engagement  
pages_show_list
```

These are important because:

pages_manage_posts

Allows Page posts to be created/managed.

pages_read_engagement

Allows Page content/engagement access.

pages_show_list

Allows access to the list of Pages the user manages.

The current Meta Facebook API Postman collection uses Page access tokens and these Page-related permissions in its Pages flow. ([Postman]^[3])

PART 19 — Facebook Login for Business

Inside the Meta dashboard you opened:

Facebook Login for Business

Then:

Configurations

Then:

Create configuration

PART 20 — Create Facebook configuration

You named it:

Facebook Page Publishing

Then Meta asked:

Choose access token

You selected:

User access token

The configuration then contained the Page-related permissions.

The final configuration was successfully created.

PART 21 — Why there are two Facebook tokens

This is one of the most important things to understand.

You first get:

User Access Token

That represents the Facebook user.

Then you call:

```
/me/accounts
```

and Meta gives you:

```
Page Access Token
```

The Page token acts on behalf of the Page.

Meta's current Facebook API collection explicitly documents this flow: use the User Access Token with `/me/accounts` , then take the returned Page `access_token` as the Page Access Token. ([Postman]^[3])

Think:

```
Facebook User
  ↓
User Access Token
  ↓
/me/accounts
  ↓
Facebook Page
  ↓
Page Access Token
```

PART 22 — Graph API Explorer

You opened:

Graph API Explorer

from:

```
Meta Developer Dashboard
→ Testing
→ Open Graph API Explorer
```

The tool showed:

```
Meta App = VelorQ
User or Page = User Token
```

and the Page permissions.

You generated the Facebook User Access Token there.

PART 23 — Get Facebook Pages

You then called:

Method

```
GET
```

Endpoint

```
https://graph.facebook.com/v26.0/me/accounts
```

with:

```
fields=id,name,access_token,tasks
```

Conceptually:

```
GET /me/accounts
```

The official Meta Facebook collection documents:

```
/me/accounts?fields=name,access_token,tasks&access_token={user_access_token}
```

for getting the Pages managed by a user. ([Postman]^[3])

Your response gave you:

```
Page Name:  
my social publisher
```

```
Page ID:  
1315245281672428
```

```
Page Access Token:  
returned here
```

```
Tasks:
```

CREATE_CONTENT

...

That was the important step.

PART 24 — Difference between Facebook User Token and Page Token

Remember this:

```
USER ACCESS TOKEN
  ↓
belongs to Facebook user
  ↓
/me/accounts
  ↓
PAGE ACCESS TOKEN
  ↓
acts as Page
```

So when publishing to:

```
1315245281672428
```

you use the **Page Access Token**.

Not the Instagram token.

Not the Facebook User Token.

PART 25 — First Facebook text post

Before testing an image, you successfully tested Page publishing with the feed endpoint.

Method

```
POST
```

URL

```
https://graph.facebook.com/v26.0/1315245281672428/feed
```

Body

```
message = Facebook API test  
access_token = YOUR_PAGE_ACCESS_TOKEN
```

You used:

```
Authorization = No Auth
```

and placed the Page token in the request.

The successful response was:

```
{  
  "id": "122103518223475255",  
  "post_id": "1315245281672428_122103518301475255"  
}
```

That proved:

```
Facebook API  
  ↓  
Page access  
  ↓  
CREATE_CONTENT  
  ↓  
Page post
```

was working.

🎉 Facebook Page publishing was proven.

PART 26 — Your Facebook image test issue

You then tried:

```
POST /1315245281672428/photos
```

and initially used:

```
image_url
```

Facebook returned:

```
(#324) Requires upload file
```

You changed the parameter to:

```
url
```

Then you received a different error involving:

```
publish_actions
```

That was a useful debugging moment.

It told us the request/authentication path was not identical to the old examples/tutorials we were trying.

The important thing is that your **Page /feed publishing worked**, so the core Facebook Page authorization is already proven.

PART 27 — The Facebook IDs you now understand

You now have several different IDs.

App ID

Identifies your Meta application.

```
Meta App
```

Facebook Page ID

```
1315245281672428
```

Identifies:

```
my social publisher
```

Instagram User ID

```
17841433886128760
```

Identifies:

```
@velar.q
```

Instagram Container ID

Example:

```
18101283815129538
```

Temporary media container.

Facebook Post ID

Example:

```
1315245281672428_122103518301475255
```

Identifies a particular Page post.

PART 28 — All your tokens

This part is extremely important.

You have several token concepts.

Instagram access token

Used for:

Instagram API

Facebook User Access Token

Used for:

/me/accounts

to find Pages.

Facebook Page Access Token

Used for:

Page publishing

Do not treat them as interchangeable.

Instagram token

≠

Facebook User token

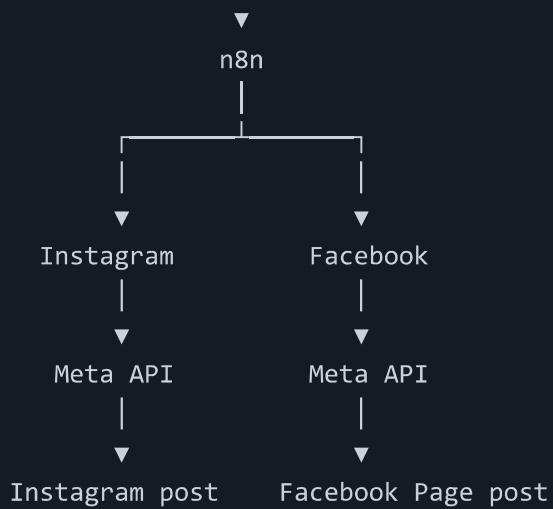
≠

Facebook Page token

PART 29 — Your final architecture

Now that both APIs work, your eventual application should look like:





PART 30 — What happens when a real user presses “Send Now”

Eventually your UI will have:

```
Upload Image  
[ photo.jpg ]  
  
Caption  
[ My new post ]  
  
Platform  
(•) Instagram  
( ) Facebook  
( ) Both  
  
[ Send Now ]
```

When they click:

Send Now

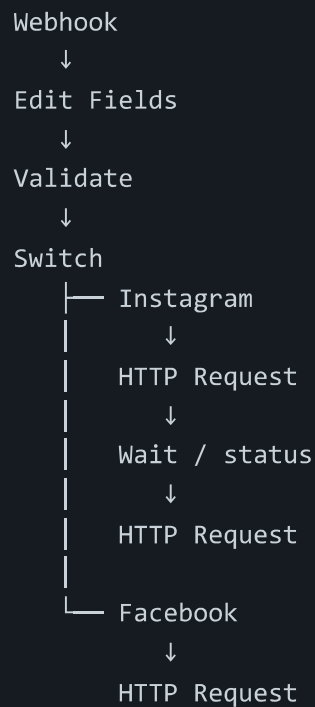
your backend sends something like:

```
{  
  "caption": "My new post",  
  "image_url": "https://storage.example.com/images/abc123.jpg",  
  "platform": "instagram"  
}
```

to n8n.

PART 31 — Your n8n workflow

The first version should be:



That directly mirrors what you already proved in Postman.

PART 32 — Instagram inside n8n

The Instagram branch will eventually contain:

Node 1

HTTP Request

Method

POST

Endpoint

```
https://graph.instagram.com/v25.0/{{ig_user_id}}/media
```

Parameters:

```
image_url  
caption  
access_token
```

Then the response gives:

```
{  
  "id": "CONTAINER_ID"  
}
```

PART 33 — Instagram status node

Next:

```
HTTP Request
```

Method

```
GET
```

URL

```
https://graph.instagram.com/v25.0/{{container_id}}
```

Query parameters:

```
fields=status_code,status  
access_token={{instagram_token}}
```

You wait until:

```
status_code = FINISHED
```

PART 34 — Instagram publish node

Then:

```
HTTP Request
```

Method

```
POST
```

URL

```
https://graph.instagram.com/v25.0/{{ig_user_id}}/media_publish
```

Body:

```
creation_id={{container_id}}  
access_token={{instagram_token}}
```

PART 35 — Facebook inside n8n

For the Page branch, the core working request you already proved is:

```
POST  
https://graph.facebook.com/v26.0/{{page_id}}/feed
```

Body:

```
message={{caption}}  
access_token={{page_access_token}}
```

For photo publishing, we'll use the current Page photo/publishing flow appropriate to the current API version rather than copying a deprecated `publish_actions` tutorial.

Meta's current Page token documentation confirms the Page access token model. ([Postman] [\[3\]](#))

PART 36 — Multiple users later

This is where your earlier question about tokens becomes very important.

You asked:

Will the token be different for different accounts?

Yes.

Eventually your database might look conceptually like:

User	Instagram ID	Instagram token	Facebook Page ID	Page token
User A	IG-A	Token-A	Page-A	PageToken-A
User B	IG-B	Token-B	Page-B	PageToken-B
User C	IG-C	Token-C	Page-C	PageToken-C

Your backend must know:

```
Which user is sending?  
    ↓  
Which connected social account?  
    ↓  
Which token?
```

PART 37 — The exact token architecture for your application

Don't build this:

```
Frontend  
    ↓  
Instagram token
```

or:

```
Frontend  
    ↓  
Facebook Page token
```

Instead:

```
Frontend
  ↓
Backend
  ↓
Database / encrypted credential store
  ↓
n8n
  ↓
Meta API
```

Tokens should never be exposed to the browser.

PART 38 — Your completed checklist

Meta / App

- ☒ Meta Developer account
- ☒ Meta App
- ☒ App settings
- ☒ App roles

Instagram

- ☒ Instagram Professional account
- ☒ Instagram API use case
- ☒ Instagram Tester
- ☒ Tester invitation accepted
- ☒ Instagram account added
- ☒ Instagram permissions
- ☒ instagram_content_publish
- ☒ Instagram access token
- ☒ Instagram numeric User ID
- ☒ Instagram media container
- ☒ Instagram status check
- ☒ Instagram media_publish
- ☒ First Instagram post

Facebook

- ☑ Facebook Page
- ☑ Page use case
- ☑ pages_manage_posts
- ☑ pages_read_engagement
- ☑ pages_show_list
- ☑ Facebook Login for Business
- ☑ Facebook configuration
- ☑ User access token
- ☑ /me/accounts
- ☑ Page ID
- ☑ Page access token
- ☑ Facebook Page post

So at this point:

Instagram API publishing = proven 

Facebook Page publishing = proven 

PART 39 — The URL cheat sheet

Here is the compact API reference you can keep.

Instagram

Create media

```
POST
https://graph.instagram.com/v25.0/{ig-user-id}/media
```

Check media

```
GET
https://graph.instagram.com/v25.0/{container-id}?fields=status_code,status
```

Publish

```
POST
https://graph.instagram.com/v25.0/{ig-user-id}/media_publish
```

Facebook

Get managed Pages

```
GET
https://graph.facebook.com/v26.0/me/accounts
```

Typical fields:

```
name
access_token
tasks
```

Meta documents this `/me/accounts` flow as the way to retrieve Page access tokens. ([Postman]^[3])

Get one specific Page

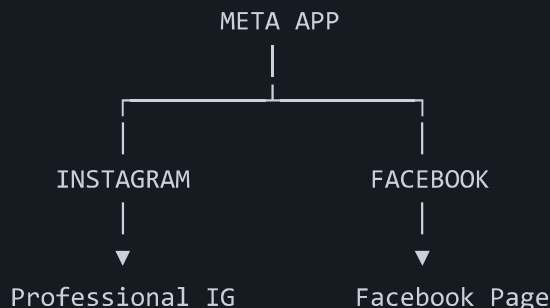
```
GET
https://graph.facebook.com/v26.0/{page-id}?fields=name,access_token
```

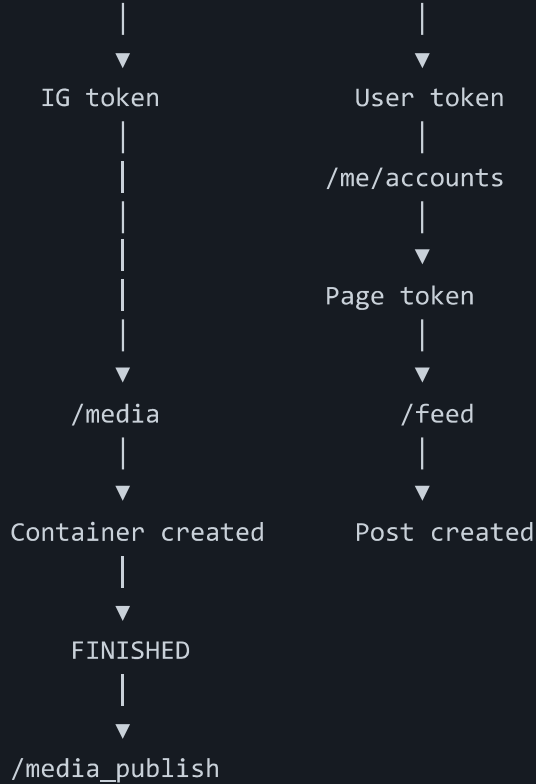
Meta's current Facebook API collection documents this direct Page-token lookup as well. ([Postman]^[4])

Create a Page text post

```
POST
https://graph.facebook.com/v26.0/{page-id}/feed
```

PART 40 — The one picture you should remember





PART 41 — What you have actually achieved

You are no longer at the “Meta setup” stage.

You have reached:

Meta API credentials



Manual API testing



Instagram post



Facebook Page post



NEXT

n8n automation

And that is the right point to start building your actual application.

Sources

Meta's official Postman workspace is the maintained API reference for these current collections. ([Postman]^[5])

Instagram Login permissions currently documented by Meta's Instagram API collection include `instagram_business_basic` and `instagram_business_content_publish`. ([Postman]^[1])

Meta's current Facebook API collection documents the User Access Token → `/me/accounts` → Page Access Token flow. ([Postman]^[3])

The next logical phase is to build your **n8n workflow pin-by-pin**, starting with the **Webhook node**, then the Instagram 3-step chain, Facebook branch, and finally the **Both** switch.